

TALLER PRÁCTICO: CRUD CON SQLITE Y FRAMEWORK PYTHON

1. INFORMACIÓN GENERAL

Fecha: 20 de junio de 2026

Duración: 4 horas

Modalidad: Individual o en parejas

Unidad Curricular: Bases de Datos I

Docente: Ing. Jesus Gregorio Altuve Roa

2. OBJETIVOS DE APRENDIZAJE

Al finalizar este taller, el estudiante será capaz de:

- Construir una API RESTful completa con operaciones CRUD utilizando un framework Python.
- Diseñar e implementar un esquema de base de datos relacional en SQLite con integridad referencial.
- Aplicar validaciones de datos robustas que garanticen la consistencia de la información.
- Manejar errores de base de datos y transacciones de manera adecuada.
- Exponer la aplicación localmente utilizando túneles de Cloudflare para pruebas remotas.

3. DESCRIPCIÓN DEL PROBLEMA

Debes construir un Sistema de Gestión de Biblioteca que permita administrar libros, autores y préstamos. La aplicación debe exponer endpoints REST para realizar operaciones CRUD sobre las entidades, aplicando validaciones estrictas y manejando relaciones entre tablas.

4. MODELO DE DATOS (ESQUEMA RELACIONAL)

El sistema debe contener al menos las siguientes entidades con sus relaciones:

- Tabla: autores

id: INTEGER PRIMARY KEY AUTOINCREMENT
 nombre: VARCHAR(100) NOT NULL
 nacionalidad: VARCHAR(50)
 fecha_nacimiento: DATE

- Tabla: libros

id: INTEGER PRIMARY KEY AUTOINCREMENT
 titulo: VARCHAR(200) NOT NULL
 isbn: VARCHAR(13) UNIQUE NOT NULL
 genero: VARCHAR(50)
 anio_publicacion: INTEGER CHECK(anio_publicacion > 1000 AND anio_publicacion <= 2026)
 autor_id: INTEGER NOT NULL, FOREIGN KEY REFERENCES autores(id) ON DELETE RESTRICT

- Tabla: usuarios

id: INTEGER PRIMARY KEY AUTOINCREMENT
 nombre: VARCHAR(100) NOT NULL
 email: VARCHAR(100) UNIQUE NOT NULL
 telefono: VARCHAR(20)

- Tabla: prestamos

id: INTEGER PRIMARY KEY AUTOINCREMENT
 libro_id: INTEGER NOT NULL, FOREIGN KEY REFERENCES libros(id)
 usuario_id: INTEGER NOT NULL, FOREIGN KEY REFERENCES usuarios(id)
 fecha_prestamo: DATE NOT NULL DEFAULT CURRENT_DATE
 fecha_devolucion: DATE
 estado: VARCHAR(20) DEFAULT "activo" CHECK(estado IN ("activo", "devuelto", "vencido"))

Restricciones de integridad a implementar:

- No se puede eliminar un autor que tenga libros asociados (ON DELETE RESTRICT).
- No se puede registrar un préstamo de un libro que ya esté prestado (estado = "activo").
- El ISBN debe contener exactamente 13 dígitos numéricos.
- El email debe tener formato válido.
- El año de publicación debe estar entre 1000 y el año actual.

5. REQUISITOS TÉCNICOS

Framework: Libre elección (FastAPI, Flask, Django REST Framework, Bottle, etc.)

Base de datos: SQLite (archivo .db local)

ORM recomendado: SQLAlchemy (opcional pero recomendado)

Validación: Pydantic (si usas FastAPI) o validaciones manuales robustas

Túnel: cloudflared (trycloudflare)

5.1 Endpoints Requeridos

Todos los endpoints deben devolver respuestas JSON con códigos HTTP apropiados.

Método	Endpoint	Descripción	Códigos HTTP
GET	/autores	Listar todos los autores (con paginación opcional)	200
GET	/autores/{id}	Obtener un autor por ID	200, 404
POST	/autores	Crear un nuevo autor	201, 400, 422
PUT	/autores/{id}	Actualizar un autor existente	200, 404, 400
DELETE	/autores/{id}	Eliminar un autor (solo si no tiene libros)	204, 404, 409
GET	/libros	Listar todos los libros (incluir nombre del autor)	200
GET	/libros/{id}	Obtener un libro por ID	200, 404
POST	/libros	Crear un nuevo libro	201, 400, 422
PUT	/libros/{id}	Actualizar un libro existente	200, 404, 400
DELETE	/libros/{id}	Eliminar un libro	204, 404
GET	/usuarios	Listar todos los usuarios	200
POST	/usuarios	Crear un nuevo usuario	201, 400, 422
GET	/prestamos	Listar todos los préstamos	200
POST	/prestamos	Registrar un nuevo préstamo	201, 400, 409
PUT	/prestamos/{id}/devolver	Marcar un préstamo como devuelto	200, 404

6. VALIDACIONES OBLIGATORIAS

El sistema debe rechazar solicitudes con datos inválidos ANTES de tocar la base de datos. Las validaciones deben incluir:

- Campos obligatorios: No permitir NULL en columnas NOT NULL.
- Tipos de datos correctos: El año de publicación debe ser entero, el email debe ser string válido.
- Longitudes máximas: Respetar los límites de VARCHAR definidos en el esquema.
- Formatos específicos: ISBN de 13 dígitos, email con formato @dominio.ext.
- Restricciones de negocio: No prestar libro ya prestado, no borrar autor con libros.
- Valores de enumeración: El estado del préstamo solo puede ser "activo", "devuelto" o "vencido".

7. MANEJO DE ERRORES Y TRANSACCIONES

Implementar manejo de excepciones para los siguientes escenarios:

- Violación de UNIQUE (ISBN duplicado, email duplicado): Devolver HTTP 409 Conflict con mensaje descriptivo.
- Violación de FOREIGN KEY (autor_id inexistente): Devolver HTTP 400 Bad Request.
- Violación de CHECK (año inválido): Devolver HTTP 422 Unprocessable Entity.
- Recurso no encontrado: Devolver HTTP 404 Not Found.
- Para operaciones que involucren múltiples pasos (ej: crear préstamo + verificar disponibilidad), usar transacciones con COMMIT/ROLLBACK.

8. EXPOSICIÓN LOCAL CON TRYCLOUDFLARE

Al finalizar el desarrollo, debes exponer tu aplicación local para que el docente pueda probarla remotamente.

Paso 1: Asegúrate de que tu aplicación corra en un puerto local (ej: `http://localhost:8000`).

Paso 2: Instala cloudflared si no lo tienes:

```
pip install cloudflared # o descarga desde https://developers.cloudflare.com/cloudflare-one/connections/connect-apps/install-and-setup/installation/
```

Paso 3: Ejecuta el túnel:

```
cloudflared tunnel --url http://localhost:8000
```

Paso 4: Copia la URL generada (ej: `https://tus-proyecto.trycloudflare.com`) y compártela con el docente.

Nota: La URL es temporal. Mantén el túnel activo durante la evaluación.

9. RÚBRICA DE EVALUACIÓN

Criterio	Descripción	Peso	Puntuación
Funcionalidad CRUD	Todos los endpoints CRUD funcionan correctamente para las 4 entidades.	25%	
Validaciones	Validaciones robustas que rechazan datos inválidos antes de la DB. Mensajes claros.	20%	
Modelo Relacional	Esquema correcto con FKs, constraints y relaciones bien definidas.	15%	
Manejo de Errores	Manejo apropiado de excepciones, códigos HTTP correctos, rollback en transacciones.	15%	
Tests / Documentación	Al menos 3 tests unitarios y documentación de endpoints (Swagger o README).	10%	
Exposición Funcional	La aplicación está accesible vía trycloudflare y responde correctamente.	10%	
Código Limpio	Estructura modular, nombres descriptivos, sin código duplicado.	5%	

Escala de calificación: 0-20 puntos, donde 20 = Excelente, 15 = Bueno, 10 = Regular, 5 = Deficiente, 0 = No cumple.

10. ENTREGABLES

- Repositorio en GitHub con el código fuente completo.
- Archivo README.md con instrucciones de instalación y ejecución.
- URL de trycloudflare activa al momento de la evaluación.
- Capturas de pantalla de las pruebas realizadas (Postman, curl, o navegador).

11. NOTAS IMPORTANTES

- No se permite el uso de herramientas de IA generativa para escribir el código. Se evaluará la comprensión individual durante la defensa.
- El plagio entre estudiantes resultará en nota cero (0) para todos los involucrados.
- Se valorará la creatividad en la solución y la proactividad en consultar dudas.
- El docente realizará pruebas de estrés enviando datos inválidos intencionalmente.